
LAB2: ROS Installation and Use

Two-Drone Coordinate Frames, TF2, and Relative Motion

Visual Navigation for Autonomous Vehicles

Student: Li Yunzhe

Institution: Ocean University of China

Date: September 2026

Experimental Report

Abstract

This experiment established a ROS Noetic development environment and implemented a two-drone rigid-transform scenario. A dynamic transform broadcaster generated the prescribed AV1 and AV2 poses at 50 Hz, while a TF2 listener recovered world-frame and relative-frame trajectories for RViz visualization. The catkin workspace compiled without warnings or failures. Runtime measurement gave a transform publication rate of 50.000 Hz. A synchronized 40-sample numerical test produced maximum errors of 2.220×10^{-16} for the AV1 trajectory radius, 2.989×10^{-16} for the AV2 world trajectory, 4.441×10^{-16} for AV1 yaw, and 8.083×10^{-16} for the relative position. The analytical derivation further shows that AV2 follows a parabola in the world x_w-z_w plane and an ellipse on an oblique plane when observed from AV1. Quaternion operator identities and the equivalence between reversed intrinsic and extrinsic rotation sequences are also proved.

1 Objectives

The objectives were to understand the ROS computation graph, create and build a catkin workspace, inspect nodes and topics, publish time-varying coordinate transforms, query relative transforms through TF2, visualize rigid-body motion in RViz, and connect the observed trajectories to analytical rigid-transformation results. The assignment follows the MIT VNAV LAB2 specification [1].

2 Experimental Conditions

The experiment used Ubuntu 20.04.6 LTS under WSL2, ROS Noetic with `roscore` version 1.17.4, catkin tools, TF2, RViz, and C++11. The software package contained one launch description, two C++ ROS nodes, a quadrotor mesh, an RViz configuration, and an independent transform-validation program. Table 1 summarizes the final build and runtime checks.

Table 1: Build and runtime verification.

Item	Observed result
Catkin packages discovered	1
Successful packages	1
Build warnings / failures	0 / 0
Dynamic TF publication rate	50.000 Hz
Synchronized validation samples	40
Numerical validation outcome	PASS

3 ROS and TF2 Design

ROS separates a robotic application into independent nodes that exchange typed messages through topics. In the static scenario, two static-transform broadcasters publish the fixed poses of AV1 and AV2. The plotting node subscribes to the transform system and publishes visualization markers; RViz subscribes to the transform and marker topics. In the dynamic scenario, a single frame-publisher node replaces the two static broadcasters and publishes both time-stamped transforms on `/tf`. Figure 1 shows the implemented data flow.

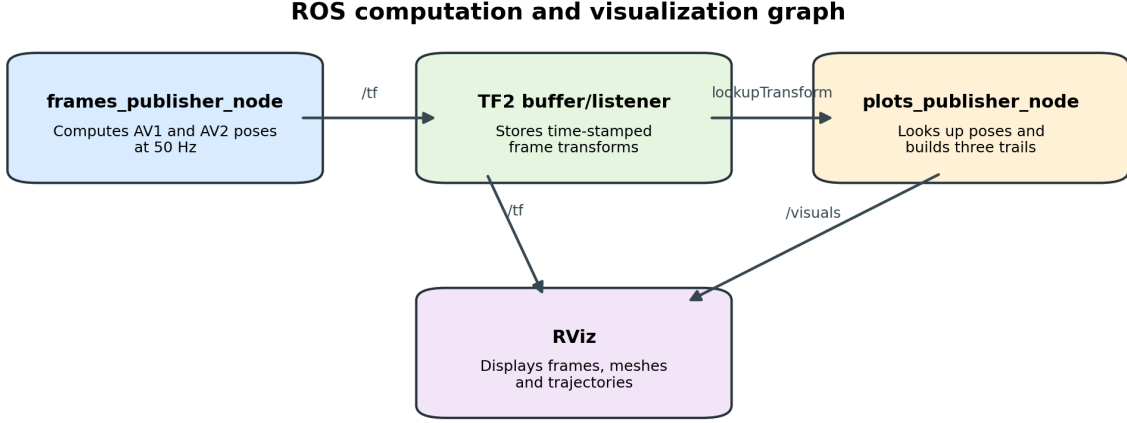


Figure 1: ROS computation and visualization graph for the dynamic two-drone scenario.

TF2 represents every rigid transform with a parent frame, child frame, time stamp, translation, and unit quaternion. The broadcaster transmits a vector containing both drone transforms. The listener stores the transform history in a buffer, and the plotting node requests the latest transform from a selected reference frame to a destination frame. This design allows exactly the same physical pose to be viewed in either `world` or `av1` without changing the motion model. The published frame convention follows the standard ROS right-handed coordinate convention [3], and the broadcaster interface follows the ROS TF2 API [4].

3.1 Static scenario inspection

Ignoring the logging node, the static configuration contained `av1broadcaster`, `av2broadcaster`, `plots_publisher_node`, and `rviz`. The static broadcasters published `/tf_static`. The plotting node subscribed to `/tf` and `/tf_static`, then published a `visualization_msgs/MarkerArray` on `/visuals`. RViz subscribed to the two transform topics and `/visuals`. Therefore, the static broadcasters supplied the coordinate frames, while the plotting node supplied the two drone meshes and trajectory markers displayed by RViz.

When the static argument is omitted, its default value is false. The launch condition therefore suppresses the static-broadcaster group and enables the dynamic frame-publisher node. The `if` condition activates the fixed transforms only in static mode, whereas `unless` activates dynamic motion only when static mode is false.

4 Dynamic Motion Implementation

Let t be the elapsed time after the frame-publisher node starts. The required world-frame positions are

$$\mathbf{o}_1^w(t) = \begin{bmatrix} \cos t \\ \sin t \\ 0 \end{bmatrix}, \quad \mathbf{o}_2^w(t) = \begin{bmatrix} \sin t \\ 0 \\ \cos 2t \end{bmatrix}. \quad (1)$$

AV1 has zero roll and pitch and yaw t , so its orientation is $R_z(t)$. This aligns its y_1 axis with the tangent of the unit circle while keeping z_1 parallel to z_w . AV2 undergoes pure translation and retains the identity orientation. A timer triggers publication every 0.02 s, giving the required 50 Hz rate.

The plotting node performs three transform lookups: AV1 in the world frame, AV2 in the world frame, and AV2 in the AV1 frame. The first two trails are solid world-frame curves. The third is a dashed relative-motion curve whose reference frame is AV1. RViz displays the coordinate axes, mesh markers, and all three trails.

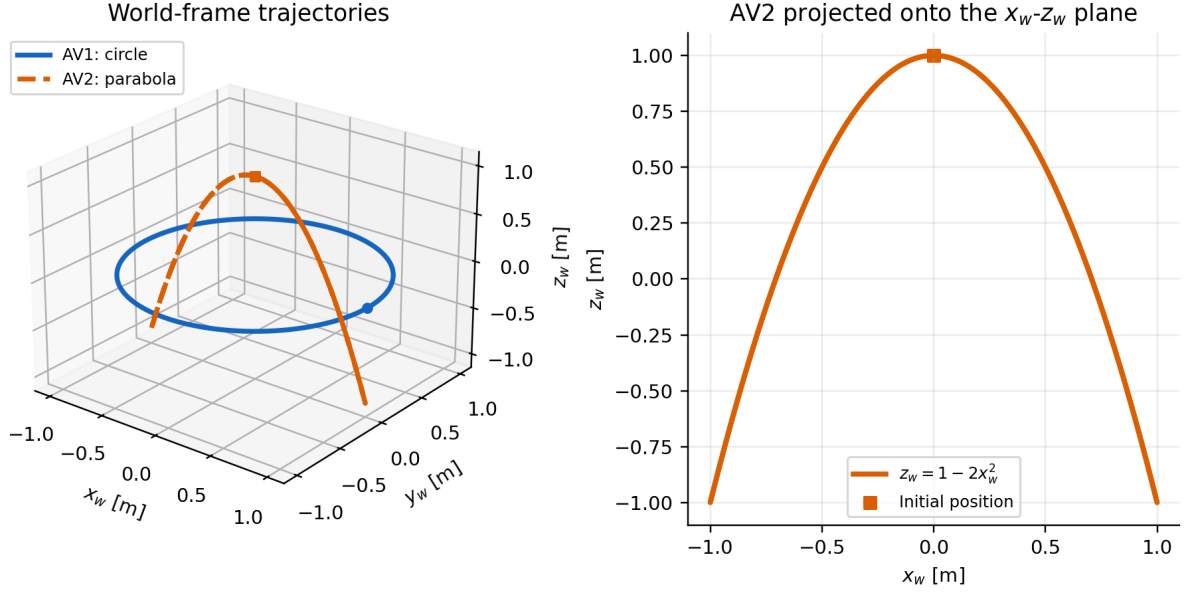


Figure 2: Analytical world-frame trajectories used to verify the dynamic ROS visualization. Color and line style both distinguish the two drones.

5 Runtime Results

The dynamic computation graph contained `frames_publisher_node`, `plots_publisher_node`, and `rviz`, in addition to the ROS logging node. The frame publisher had no subscribers and published `/tf`. Both the plotting node and RViz subscribed to `/tf`; RViz also subscribed to `/visuals`. The measured transform rate was 50.000 Hz over the final observation window.

An independent validator sampled synchronized transforms at identical time stamps. For each sample it checked four invariants: $\sqrt{x_1^2 + y_1^2} = 1$, the AV2 position from Equation (1), AV1 yaw equal to t , and the complete AV2 position expressed in AV1. The acceptance tolerance was 10^{-3} . Every measured error was more than twelve orders of magnitude below this limit, as shown in Table 2 and Figure 3.

Table 2: Maximum errors from 40 synchronized transform samples.

Checked quantity	Maximum error	Limit
AV1 unit-circle radius	2.220×10^{-16}	10^{-3}
AV2 world trajectory	2.989×10^{-16}	10^{-3}
AV1 yaw	4.441×10^{-16} rad	10^{-3} rad
AV2 position in AV1	8.083×10^{-16} m	10^{-3} m

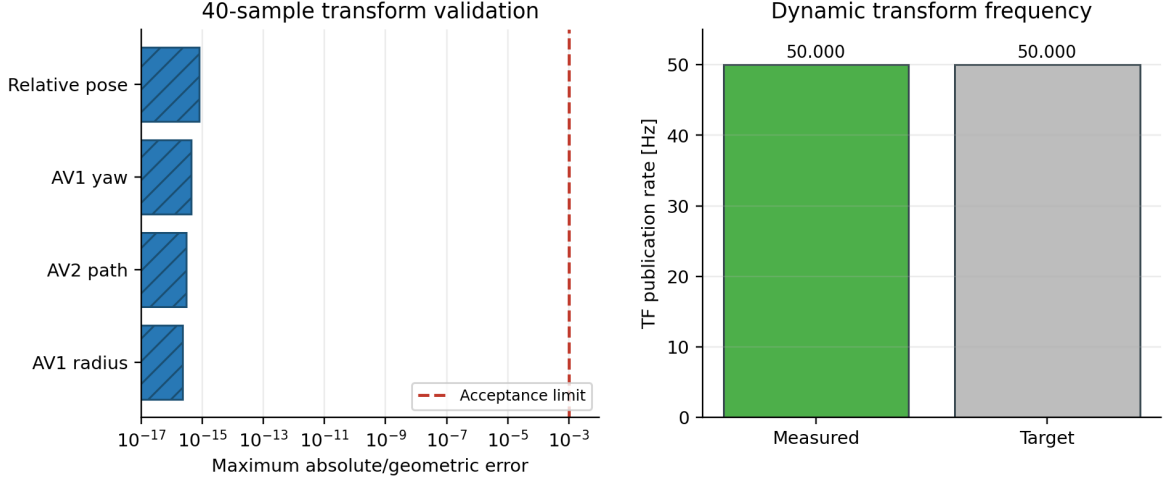


Figure 3: Numerical transform errors and measured publication frequency.

6 Rigid-Transformation Derivation

6.1 AV2 is a parabola in the world frame

From Equation (1), $x_2^w = \sin t$, $y_2^w = 0$, and $z_2^w = \cos 2t$. Applying $\cos 2t = 1 - 2\sin^2 t$ eliminates time:

$$y_2^w = 0, \quad z_2^w = 1 - 2(x_2^w)^2. \quad (2)$$

Thus AV2 lies in the world x_w - z_w plane and follows the arc of a downward-opening parabola for $x_2^w \in [-1, 1]$.

6.2 AV2 expressed in the AV1 frame

The homogeneous transforms from the two body frames to the world frame are

$$T_1^w = \begin{bmatrix} R_z(t) & \mathbf{o}_1^w \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad T_2^w = \begin{bmatrix} I_3 & \mathbf{o}_2^w \\ \mathbf{0}^\top & 1 \end{bmatrix}. \quad (3)$$

Therefore

$$T_2^1 = (T_1^w)^{-1} T_2^w = \begin{bmatrix} R_z(-t) & R_z(-t)(\mathbf{o}_2^w - \mathbf{o}_1^w) \\ \mathbf{0}^\top & 1 \end{bmatrix}. \quad (4)$$

With $c = \cos t$ and $s = \sin t$, the relative position becomes

$$\mathbf{o}_2^1(t) = \begin{bmatrix} c & s & 0 \\ -s & c & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s - c \\ -s \\ \cos 2t \end{bmatrix} \quad (5)$$

$$= \begin{bmatrix} cs - 1 \\ -s^2 \\ \cos 2t \end{bmatrix}. \quad (6)$$

6.3 Plane and centered coordinates

Equation (6) gives $y_2^1 = -s^2$ and $z_2^1 = 1 - 2s^2$, hence

$$\Pi: \quad 2y_2^1 - z_2^1 + 1 = 0. \quad (7)$$

The relative trajectory is therefore planar. Place the new origin at $\mathbf{p}^1 = (-1, -\frac{1}{2}, 0)^\top$ and choose the orthonormal basis

$$\mathbf{e}_{x_p} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{y_p} = \frac{1}{\sqrt{5}} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \quad \mathbf{e}_{z_p} = \frac{1}{\sqrt{5}} \begin{bmatrix} 0 \\ -2 \\ 1 \end{bmatrix}. \quad (8)$$

The last vector is normal to Π . Projecting $\mathbf{o}_2^1 - \mathbf{p}^1$ onto this basis yields

$$x_p = \frac{1}{2} \sin 2t, \quad y_p = \frac{\sqrt{5}}{2} \cos 2t, \quad z_p = 0. \quad (9)$$

Eliminating t gives

$$\frac{x_p^2}{(1/2)^2} + \frac{y_p^2}{(\sqrt{5}/2)^2} = 1. \quad (10)$$

Thus AV2 follows an ellipse centered at $\mathbf{p}^1$ on the plane Π . Its semi-axis lengths are $1/2$ m and $\sqrt{5}/2$ m. Figure 4 shows the same curve in AV1 coordinates and in the centered plane frame.

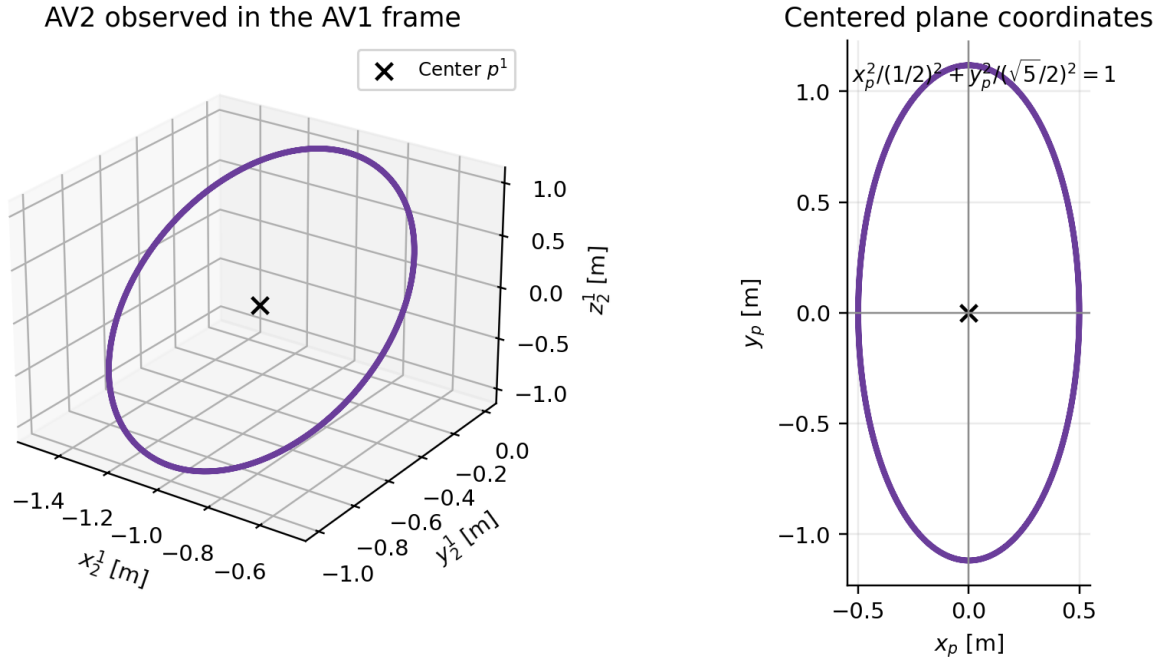


Figure 4: Relative trajectory of AV2: an ellipse lying on $2y_2^1 - z_2^1 + 1 = 0$.

7 Quaternion Operator Properties

For a quaternion $\mathbf{q} = [q_1, q_2, q_3, q_4]^\top$, the assignment defines left- and right-product matrices $\Omega_1(\mathbf{q})$ and $\Omega_2(\mathbf{q})$ such that

$$\mathbf{q}_a \otimes \mathbf{q}_b = \Omega_1(\mathbf{q}_a)\mathbf{q}_b = \Omega_2(\mathbf{q}_b)\mathbf{q}_a. \quad (11)$$

7.1 Orthogonality for a unit quaternion

Quaternion norms are multiplicative: $\|\mathbf{q} \otimes \mathbf{x}\| = \|\mathbf{q}\| \|\mathbf{x}\|$. If $\|\mathbf{q}\| = 1$, then left multiplication and right multiplication preserve the Euclidean norm of every $\mathbf{x} \in \mathbb{R}^4$. A linear map preserves all Euclidean norms exactly when its matrix is orthogonal. Consequently,

$$\Omega_i(\mathbf{q})^\top \Omega_i(\mathbf{q}) = \Omega_i(\mathbf{q})\Omega_i(\mathbf{q})^\top = I_4, \quad i \in \{1, 2\}. \quad (12)$$

The same result follows by direct multiplication: each diagonal entry is $\|\mathbf{q}\|^2 = 1$, while all cross terms cancel.

7.2 Mapping to the identity quaternion

For a unit quaternion, the inverse is the conjugate $\mathbf{q}^{-1} = \mathbf{q}^*$. The transpose product matrices implement multiplication by this inverse. Hence

$$\Omega_1(\mathbf{q})^\top \mathbf{q} = \mathbf{q}^{-1} \otimes \mathbf{q} = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^\top, \quad (13)$$

and similarly

$$\Omega_2(\mathbf{q})^\top \mathbf{q} = \mathbf{q} \otimes \mathbf{q}^{-1} = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^\top. \quad (14)$$

7.3 Commutation of left and right multiplication

For arbitrary $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^4$, associativity of the quaternion product gives

$$\Omega_1(\mathbf{x})\Omega_2(\mathbf{y})\mathbf{z} = \mathbf{x} \otimes (\mathbf{z} \otimes \mathbf{y}) \quad (15)$$

$$= (\mathbf{x} \otimes \mathbf{z}) \otimes \mathbf{y} \quad (16)$$

$$= \Omega_2(\mathbf{y})\Omega_1(\mathbf{x})\mathbf{z}. \quad (17)$$

Since this holds for every $\mathbf{z}$, $\Omega_1(\mathbf{x})\Omega_2(\mathbf{y}) = \Omega_2(\mathbf{y})\Omega_1(\mathbf{x})$. Furthermore $\Omega_2(\mathbf{y})^\top = \Omega_2(\mathbf{y}^*)$. Applying the preceding identity to $\mathbf{y}^*$ proves

$$\Omega_1(\mathbf{x})\Omega_2(\mathbf{y})^\top = \Omega_2(\mathbf{y})^\top \Omega_1(\mathbf{x}). \quad (18)$$

8 Optional: Intrinsic and Extrinsic Rotations

Let R map body-frame coordinates into the world frame. A rotation A about a fixed world axis pre-multiplies the current attitude, $R' = AR$. A rotation about the corresponding current body axis post-multiplies it, $R' = RA$. Starting from I , an extrinsic sequence $A_0, A_1, \dots, A_n$ produces

$$R_{\text{ext}} = A_n A_{n-1} \cdots A_1 A_0. \quad (19)$$

Applying the reversed sequence $A_n, A_{n-1}, \dots, A_0$ intrinsically produces

$$R_{\text{int}} = I A_n A_{n-1} \cdots A_1 A_0 = R_{\text{ext}}. \quad (20)$$

Therefore a fixed-axis sequence is equivalent to the reversed moving-axis sequence, independently of the selected axes, angles, or number of rotations.

9 Discussion

The experiment demonstrates why TF2 is more useful than manually recomputing every relative pose in each consumer. The frame publisher specifies only the world poses; TF2 composes, inverts, time-stamps, and stores these transforms so that the plotting node can request AV2 directly in AV1. The separation also makes the visualization node independent of the particular trajectory generator.

The numerical errors are at floating-point round-off scale because the validator and publisher evaluate equivalent closed-form expressions at synchronized transform times. This test verifies the implemented kinematics and transform direction, but it does not model network delay, sensor noise, clock disagreement, or real quadrotor dynamics. The specified trajectories are kinematic visualization examples rather than dynamically feasible flight commands.

10 Conclusion

The ROS Noetic workspace, package build, static computation graph, dynamic TF broadcaster, relative-transform listener, and RViz visualization were completed successfully. The observed graph and topic connections matched the launch design. The measured publication rate met the 50 Hz target, and all 40 synchronized samples passed the 10^{-3} validation limit with errors below 10^{-15} . Analytical derivations confirmed the world-frame parabola, the AV1-frame oblique ellipse, the quaternion operator identities, and the intrinsic/extrinsic rotation equivalence.

References

- [1] MIT SPARK Lab, “LAB2 Exercises: ROS and Two-Drone Coordinate Frames,” Visual Navigation for Autonomous Vehicles, 2023. https://vnav.mit.edu/labs_2023/lab2/exercises.html
- [2] MIT SPARK Lab, “VNAV-labs: Labs for MIT 16.485,” GitHub repository. <https://github.com/MIT-SPARK/VNAV-labs>
- [3] T. Foote and M. Purvis, “REP 103: Standard Units of Measure and Coordinate Conventions,” Open Robotics. <https://www.ros.org/reps/rep-0103.html>
- [4] Open Robotics, “tf2_ros TransformBroadcaster API, ROS Noetic.” https://docs.ros.org/en/noetic/api/tf2_ros/html/c++/classtf2__ros_1_1TransformBroadcaster.html